

Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Programación Python con Aplicaciones en el Ámbito Científico

Profesor Alan Badillo Salas

Tarea 1

Introducción a Python

Semana 1. Lunes 31, agosto 2026.

1. Introducción

En la primera sesión del curso introducimos la programación en Python como resultado de un modelado lógico de algoritmos mediante **Diagramas de Flujo**, los cuales se componen de símbolos visuales que permiten expresar la lógica paso a paso desde un inicio y hasta un fin. El diagrama luego es traducido a **Pseudocódigo** usando el lenguaje natural y marcando cada paso como un punto numerado. Finalmente, este pseudocódigo es traducido a **Código Python**, para ser ejecutado y validado.

En esta **Tarea 1**, seguiremos desarrollando las habilidades de modelado de programas y algoritmos desde el diagrama de flujo, el pseudocódigo y finalmente el código de Python.

Lee atentamente cada problema y resuélvelo. En un documento **PDF** con una portada breve la identidad del *Instituto Politécnico Nacional*, el *Centro de Investigación en Computación* y el *Departamento de diplomados y extensión profesional*, con tu nombre completo, correo, así como lugar y fecha, documenta los resultados de la tarea y envíalos por correo a **badillosalas@outlook.com** con el título **PyCien 2026 - Tarea 1**. No es necesario poner cuerpo en el correo o adjuntar más archivos, solo el documento PDF es suficiente. El documento PDF se debe llamar *pycien26.SIGLAS.pdf* con las SIGLAS formadas por tu nombre y apellidos.

1.1. Problema 1 - Inflación Simple

Diseña un diagrama de flujo, pseudocódigo y el código de python, para un programa que defina dos precios y calcule la inflación dada por la siguiente ecuación:

$$inflacion = \frac{precio_{actual} - precio_{anterior}}{precio_{anterior}}$$

La inflación mide el cambio en el precio de un producto o bien, y determina cuántas veces crece la diferencia entre el precio actual del producto y el precio anterior, por ejemplo, ¿Cuál es la inflación de la leche si en 2018 costaba \$21 MXN y actualmente en 2026 cuesta \$43 MXN?

Calcula la inflación de la leche del ejemplo y determina cuántas veces ha crecido el precio en 8 años. Contesta las siguientes preguntas:

1. ¿Qué significa una inflación de 0,5 o 50 %?
2. ¿Qué significa una inflación de 1,0 o 100 %?
3. ¿Qué significa una inflación de 1,5 o 150 %?
4. Si el salario mínimo aumenta, pero al mismo tiempo la canasta básica también aumenta, ¿Qué segmento de la población se ve afectada al comprar productos de la canasta básica?

1.2. Problema 2 - Suma de los impares

Diseña un diagrama de flujo, pseudocódigo y el código de python, para un programa que sume e imprima los número pares entre 1 y 99, como en la siguiente ecuación:

$$suma = 1 + 3 + 5 + 7 + \dots + 95 + 97 + 99$$

Para resolver este problema, se pueden plantear dos formas:

1. Crea un iterador de 0 a 49 y suma $suma = suma + 2 * i + 1$
2. Crea un iterador de 1 a 99 y suma $suma = suma + i$ solo si $i \% 2 == 1$, es decir, suma solamente si el iterador módulo 2 es igual a 1 (es impar o no es divisible por 2, porque su residuo divisional es 1).

Pista: Primero deberás definir una variable *suma* que comience en cero, luego crea un iterador para cada *i* en el rango adecuado, y recuerda que en python la función *range(a, b + 1)* requiere sumarle uno más al límite final porque sino, no lo toca. Al final imprime la suma, o hazlo en cada iteración.

1.3. Problema 3 - Traducir el diagrama de flujo

El siguiente diagrama de flujo determina si un triángulo es válido dado tres lados l_1 , l_2 y l_3 verificando las siguientes reglas:

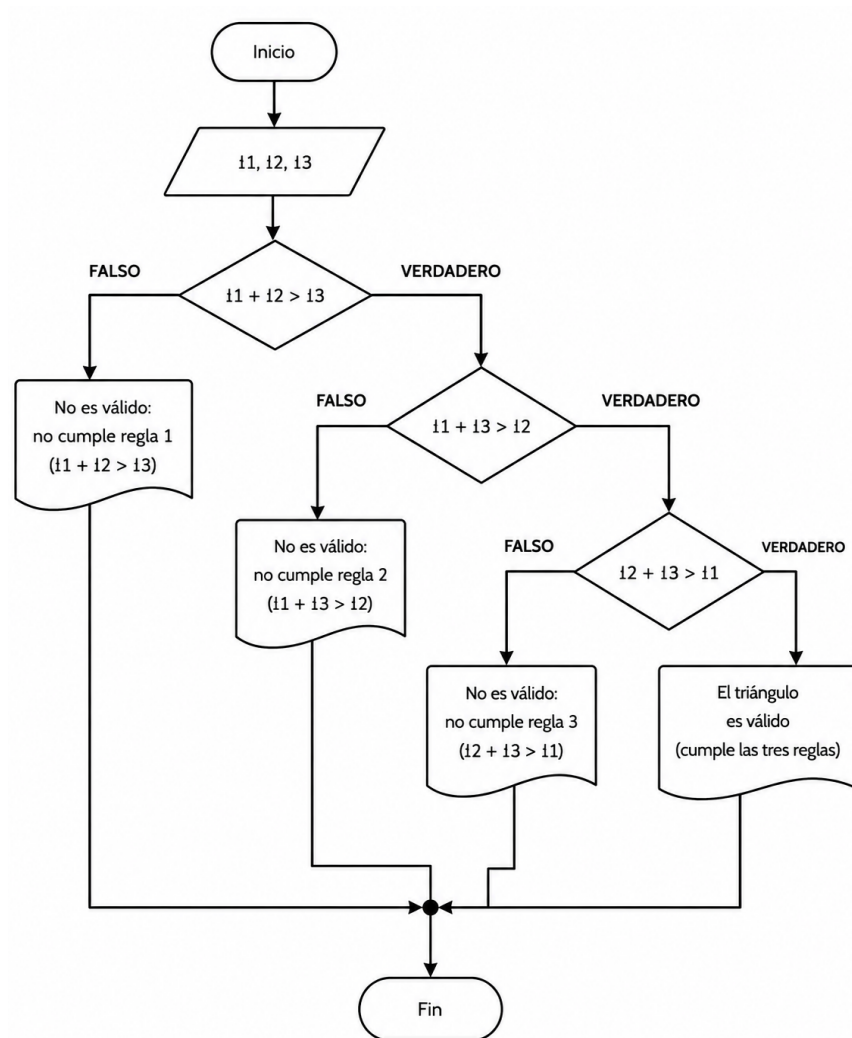
1. $l_1 + l_2 > l_3$
2. $l_1 + l_3 > l_2$
3. $l_2 + l_3 > l_1$

Solo si se cumplen las tres reglas el triángulo será válido, pero si alguna de las reglas no se cumple se debe explicar cuál regla no se cumple.

Verifica si los siguientes triángulos descritos por sus lados son válidos:

1. $l_1 = 1, l_2 = 2, l_3 = 3$
2. $l_1 = 1, l_2 = 4, l_3 = 2$
3. $l_1 = 8, l_2 = 2, l_3 = 5$
4. $l_1 = 3, l_2 = 4, l_3 = 5$

Traduce el diagrama de flujo siguiente en pseudocódigo y código de python:



Pista: Recuerda que al numerar los puntos del pseudocódigo el punto de SI (condición) tiene la misma numeración e indentación que el SINO, por ejemplo:

1. INICIO
2. SI llueve:
 - 2.1 Imprime "Está lloviendo"
2. SINO:
 - 2.2 Imprime "No llueve"
3. FIN

1.4. Problema 4 - Traducir el pseudocódigo

Genera el diagrama de flujo y el código en python a partir del siguiente pseudocódigo que calcula el área de un triángulo de lados $a = 3$, $b = 4$ y $c = 5$.

Área de un triángulo de lados a, b, c

```
1. INICIO
2. a = 3
2. b = 4
2. c = 5
2. s = (a + b + c) / 2
2. d = s * (s - a) * (s - b) * (s - c)
2. area = d ** 0.5
2. IMPRIME area
10. FIN
```

Determina el diagrama de flujo y código de python, considera que las ecuaciones para determinar el área del triángulo son:

$$s = \frac{a + b + c}{2}$$

$$area = \sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$$

Observa que si la mediatriz s es menor que solo uno de los lados, entonces la raíz cuadrada será negativa y el área será imaginaria, por lo que el triángulo debería ser válido.

Determina el área de los siguientes triángulos:

1. $a = 1, b = 1, c = 1$
2. $a = 2, b = 4, c = 6$
3. $a = 8, b = 6, c = 5$
4. $a = 9, b = 1, c = 9$
5. $a = 1, b = 2, c = 3$

1.5. Problema 5 - Traducir el código python

Genera el diagrama de flujo y el pseudocódigo a partir del siguiente código de python que ordena una lista de números mediante el algoritmo de ordenamiento burbuja.

Algoritmo de ordenamiento burbuja

Una lista de elementos, se define separando cada valor

```

# mediante comas y envolviendo todos entre corchetes []
numeros = [5, 3, 1, 2, 4, 6, 8, 0, 9, 3, 2, 1]

# El algoritmo burbuja consiste en llevar los valores más altos
# al final, recorriendo desde el primer índice de la lista
# hasta el último y comparando si el valor recorrido
# es mayor al valor observado, entonces, se intercambian

# Para lograr esto, se crean dos iteradores,
# el primer iterador i recorrerá los índices
# de 0 a n - 1, donde n es número de elementos
# en la lista, luego un segundo iterador j interno,
# comparará el valor de la lista en el índice i y j
# Si el valor en la lista de números en i, es menor
# que el valor en j, entonces, se intercambiarán
# los valores del índice i y j, para esto, una variable
# temporal guardará el valor de la lista en el índice i
# y se pondrá en este índice el valor de la lista en j,
# luego en el índice j se pondrá el valor temporal que
# era el original de la lista en la lista i.

n = len(numeros) # len(●) devuelve el total de elementos

print(f"Hay {n} elementos en la lista de números")

print(numeros)

for i in range(n): # rango: 0, 1, ..., n - 1
    for j in range(n):
        # Si el valor en el índice i es menor que en j
        # entonces, intercambia (los mayores flotan)
        if numeros[i] < numeros[j]:
            print(f"Intercambia los índices ({i}, {j})")
            print(f"Porque {numeros[i]} < {numeros[j]}")
            # Intercambio de los valores en la lista
            # entre el índice i y el j
            # Observa que retenemos el valor en i
            # porque este se perderá después de hacer
            numeros[i] = numeros[j]
            temporal = numeros[i]
            numeros[i] = numeros[j]
            numeros[j] = temporal

print(numeros)

# Al finalizar, los números quedarán ordenados, por

```

```
# lo que sería bueno ir imprimiendo cada intercambio
# y la lista de números antes de ordenarla y
# después de ordenarla
```

El resultado de ejecutar el código es:

```
Hay 12 elementos en la lista de números
[5, 3, 1, 2, 4, 6, 8, 0, 9, 3, 2, 1]
Intercambia los índices (0, 5)
Porque 5 < 6
Intercambia los índices (0, 6)
Porque 6 < 8
Intercambia los índices (0, 8)
Porque 8 < 9
Intercambia los índices (1, 0)
Porque 3 < 9
...
Intercambia los índices (11, 7)
Porque 4 < 5
Intercambia los índices (11, 8)
Porque 5 < 6
Intercambia los índices (11, 9)
Porque 6 < 8
Intercambia los índices (11, 10)
Porque 8 < 9
[0, 1, 1, 2, 2, 3, 3, 4, 5, 6, 8, 9]
```

Pista: Copia el código y ejecútalo para probar que funcione. Luego quita los comentarios que comienzan con `#` para reducir el código a unas cuantas líneas. Finalmente, lleva ese código a un pseudocódigo y después a un diagrama de flujo (ingeniería inversa). Intenta reconstruir el diagrama de flujo pensando en el problema.

Ordena las siguientes listas de números y reporta los intercambios realizados:

1. *numeros* = [1, 2, 4, 3]
2. *numeros* = [4, 3, 2, 1]
3. *numeros* = [4, 1, 3, 2]
4. *numeros* = [1, 4, 3, 2]
5. *numeros* = [4, 1, 1, 4]